

衛星データ解析技術研究会 技術セミナー（応用編）

第3回：3D技術①

スマホセンサーとAR

2026年7月3日（金）

到達目標：スマホのセンサーを Web API から直接触り、地理座標とAR空間の対応関係を把握する

時間	内容
10分	イントロ・スマホセンサーの全体像
20分	GeolocationAPI / DeviceOrientationEvent の特性と制約整理
10分	WebXR / A-Frame / AR.js のエコシステム概観
35分	ハンズオン①：Geolocation + DeviceOrientation での方位取得・確認
10分	休憩
35分	ハンズオン②：AR.js + A-Frame で観測ポイントラベルをカメラ映像にオーバーラップ
10分	デバイス差異・パーミッション周りの注意点整理
10分	振り返り・Q&A

SESSION 3 / 3D技術①

本セッションの位置づけとゴール

- 第3回・第4回で3Dを扱う。第3回はスマホセンサーとAR、第4回は点群・3DGS。第5回以降の高さデータの前段にあたる。
- 目標：スマホのセンサーを Web API から直接触り、地理座標とAR空間の対応関係を把握する。
- センサーで取得した値が座標や描画にどう変換されるかを、外部SDKに任せきりにせず追えるようにする。

本日の実装はすべてブラウザ内で完結する。ネイティブアプリ（ARKit/ARCore）は使わない。

スマホセンサーの全体像

センサー	取得する物理量	Web API	代表的な用途
GNSS（GPS等）	緯度・経度・高度・速度	Geolocation API	現在地・移動追跡
加速度センサー	3軸の加速度	DeviceMotionEvent	歩数・振動・傾き
ジャイロ스코ープ	3軸の角速度	DeviceMotionEvent（rotationRate）	姿勢変化
地磁気センサー	磁北方向	DeviceOrientationEvent	方位
カメラ	映像	getUserMedia（MediaDevices）	ARの背景映像

本日主に扱うのは *Geolocation* ・ *DeviceOrientation* ・ *getUserMedia* の3つ。

なぜブラウザから触るのか — 前提の整理

- 配布の容易さ：URLひとつで動く。アプリストアの審査・インストールが不要。
- 代償：センサーへのアクセスに制約がある（精度・権限・OS差異）。ここを理解しておくことが本日の主眼。
- 全APIに共通する前提：**Secure Context（HTTPS もしくは localhost）**でしか動かない。
- 権限モデルは分かれている：位置・カメラ・モーション（方位）はそれぞれ別に許可を取る。

SENSORS / API

Geolocation API の基礎

```
// 位置を継続的に監視する
const watchId = navigator.geolocation.watchPosition(
  (pos) => {
    const { latitude, longitude, accuracy } = pos.coords;
    // accuracy は「半径 accuracy メートルの円内にいる確度」
    console.log(latitude, longitude, accuracy);
  },
  (err) => console.error(err.code, err.message),
  { enableHighAccuracy: true, timeout: 10000, maximumAge: 0 }
);
```

- `getCurrentPosition`（1回）／`watchPosition`（継続）の2系統。
- `coords` に含まれる主な値：`latitude` / `longitude` / `accuracy` / `altitude` / `altitudeAccuracy` / `heading` / `speed`。

Geolocation の制約

- **HTTPS 必須** (Secure Context)。ローカル開発では localhost のみ例外。
- **精度は環境依存**：屋内・ビル街で大きく劣化する。 `accuracy` は誤差半径（メートル）で返るので、値として扱う。
- `heading` / `speed` は移動中のみ有効なことが多く、静止時は `null`。 **方位は Geolocation ではなく DeviceOrientation 側で取る。**
- 一度権限を拒否されると、再要求はブラウザ設定からになり難しい。最初の要求タイミングの設計が重要。

DeviceOrientationEvent の座標系

端末の姿勢を3つの角度で表す。

- **alpha**：Z軸まわりの回転。 0-360 （≒ 方位に相当）
- **beta**：X軸まわりの回転。 -180-180 （前後の傾き）
- **gamma**：Y軸まわりの回転。 -90-90 （左右の傾き）

実機で値を出しながら端末を傾けて見せると直感的に伝わる。デモ推奨箇所。

方位の取得は端末ごとに挙動が違う

同じ「北」を出すのに、iOSとAndroidで処理が異なる。

```
function extractHeading(e) {  
  if (typeof e.webkitCompassHeading === "number") {  
    // iOS: 0 = 北、時計回り。真北基準の値がそのまま得られる  
    return e.webkitCompassHeading;  
  }  
  if (e.absolute && e.alpha != null) {  
    // Android(絶対): alpha は反時計回り。方位へ変換する  
    return (360 - e.alpha) % 360;  
  }  
  return null; // 取得不可  
}
```

- **iOS の `alpha` は相対値**（リスニング開始時が基準）。真北が欲しければ `webkitCompassHeading` を使う。
- **Android の `deviceorientation` は絶対方位を保証しない**。 `deviceorientationabsolute` イベントを使う。
- 上の変換式・挙動は端末とOSで揺れる。**値を鵜呑みにせず、ハンズオン①で必ず実機検証する。**

パーミッション (iOS 13+ の追加要件)

```
async function enableOrientation() {  
  // iOS 13+ のみ requestPermission が存在する  
  if (typeof DeviceOrientationEvent.requestPermission === "function") {  
    const res = await DeviceOrientationEvent.requestPermission();  
    if (res !== "granted") return;  
  }  
  // Android はこの分岐を通らず、そのまま登録される  
  window.addEventListener("deviceorientationabsolute", extractHeading);  
  window.addEventListener("deviceorientation", extractHeading);  
}
```

- iOS では `requestPermission()` を必ず「ユーザージェスチャ（ボタンタップ等）の中」で呼ぶ。ページ読み込み時の自動実行は拒否される。
- Android は基本的に許可ダイアログ不要（ただし HTTPS は必須）。

ECOSYSTEM

3つの層 — WebXR / A-Frame / AR.js

- **WebXR Device API**：ブラウザ標準の低レベルAR/VR API。WebGLで描画。
- **A-Frame**：Three.js の上に載る宣言的HTMLフレームワーク。 `<a-scene>` などのタグで3D空間を書ける。
- **AR.js**：A-Frame等と組み合わせるWebARライブラリ。マーカーベースとロケーションベースの2系統を持つ。

「標準API」「3D記述フレームワーク」「ARの位置合わせ」で役割が分かれている、と整理すると見通しがよい。

WebXRの現状と、今回AR.jsを採用する理由

環境	WebXR immersive-ar
Android Chrome（ARCore対応機）	動作する
iPhone / iPad Safari	immersive-ar は利用できない
visionOS Safari	VRのみ。ARは非対応

- **iPhoneとAndroidの両方で同じ教材を動かしたい** → WebXRだけに依存できない。
- そこで、**getUserMedia（カメラ） + DeviceOrientation（方位）** で成立する **AR.js のロケーションベース** を実装の主軸にする。
- WebXRは「標準として概観する」位置づけにとどめる。

iOSのWebXR対応状況はOSアップデートで変わりうる領域。本日時点の前提として扱う。

AR.jsの2系統

- マーカーベース：ARマーカー（Hiroマーカー等）を基準に3Dオブジェクトを配置。屋内・展示・印刷物との連動に向く。
 - ロケーションベース：GPS + 方位で **地理座標そのもの** にオブジェクトを配置。屋外・現地調査に向く。
- 本日のハンズオン②では **ロケーションベース** を使い、観測ポイントのラベルをカメラ映像に重ねて表示する。

第2回までで扱った「地理座標を扱う」感覚が、そのままAR空間の配置につながる。

CASE STUDY

位置情報ベースARの限界 — 生駒市・鳥居ARの事例

移設前の場所に鳥居をAR表示し、一般ユーザーがスマホで体験できるようにするプロジェクト（講師が生駒市向けに実装）。

- 当初は位置情報ベースで、ジオフェンス的な配置を検討した。
- ブラウザの位置精度は数メートル程度で、cm単位の配置は成立しなかった。マーカーベースも検討したうえで、最終的に **AR.js内の座標値ベース** の配置に切り替えた。
- QRコード経由でアクセスさせ、ユーザーの立ち位置・姿勢を限定することで実用的な精度を確保した。

Demo: <https://ikominaprvj.xsrv.jp/AR/> / <https://ikoma-demo.netlify.app>

ハンズオン②のラベル表示は数メートル精度で足りる用途。cm単位が必要な場合はマーカーやVPSを検討する、という切り分けにつながる。

ハンズオン①

位置・方位の確認

- 作るもの：スマホの位置（緯度経度・精度）と方位を、Web APIから取り出して表示する一枚もの。
- 目的：iOSとAndroidで方位の取得方式が変わることを、実機で確認する。
- 開く：配布URL（`.../01_sensor/app/`）を各自のスマホで。[HTTPS必須。詳しい手順は `handson/01\_sensor` を参照。](#)

ハンズオン① 手順

1. 配布URLを開く（iPhoneは Safari、Androidは Chrome）。
2. 「センサーを開始」を押す。iPhoneは許可を選ぶ（`requestPermission` をタップ内で呼んでいる）。
3. 位置（緯度・経度・精度）が表示されることを確認する。
4. 端末を水平に持って一回転し、方位ダイヤルの針が北を指すか確認する。
5. 傾けて β / γ の変化を見る。

確認点：取得方式の表示（iOS = `webkitCompassHeading`、Android = `deviceorientationabsolute`）、横持ちでのズレ。

観察：方位の生値はそのまま使えない

- 針が北を指さないことがある → 地磁気センサーの未キャリブレーション（端末を8の字に振る）。
 - 横持ちで方位がずれる → 画面回転（`screen.orientation.angle`）ぶんの補正が要る。
 - iOS の alpha は相対、Android は `deviceorientationabsolute` で絶対。
- センサーの値と、実際に使える方位の間に補正が挟まる。ここを実機で体感する。

休憩（10分）

再開後はハンズオン②。屋外に移動できる準備をしておく。

ハンズオン②

観測ポイントを重ねて表示する

- 作るもの：周囲の観測ポイント（ラベル）と圃場区画（押し出したブロック）を、カメラ映像に重ねて表示する。
- 使うもの：A-Frame 1.6.0 + AR.js 3.4.7（ロケーションベース）、`observation_points.geojson`。
- 開く：配布URL（`.../02_ar/app/`）。屋外で、HTTPS必須。NDVIで色分け（水域は青、植生が濃いほど緑）。

AR.js ロケーションベースの構成

```
<script src="https://aframe.io/releases/1.6.0/aframe.min.js"></script>
<!-- raw.githack 経由で配信する（raw.githubusercontent の直リンクはMIME制限で実行されない） -->
<script src="https://raw.githack.com/AR-js-org/AR.js/3.4.7/aframe/build/aframe-ar.js"></script>

<a-scene arjs="sourceType: webcam; videoTexture: true; debugUIEnabled: false;">
  <a-camera look-controls-enabled="false"
    arjs-device-orientation-controls="smoothingFactor: 0.1"
    gps-new-camera="gpsMinDistance: 5"></a-camera>
</a-scene>
```

- `gps-new-camera` : GPSの現在地をワールド座標に対応づけるカメラ。
- `gps-new-entity-place="latitude: ...; longitude: ..."` : 緯度経度にオブジェクトを配置する。
- 新ロケーションベース（`gps-new-*`）が現行の推奨。AR.js 3.4.7 は A-Frame 1.6.0 を要求する。

GeoJSONから観測ポイントを配置

```
const gj = await (await fetch("./observation_points.geojson")).json();
gj.features.forEach((f) => {
  const p = f.properties;
  if (f.geometry.type === "Point") {
    const [lon, lat] = f.geometry.coordinates;
    // gps-new-entity-place を持つ a-entity を生成し、色・ラベルを付けて追加する
  }
});
```

- Point → NDVIで色分けしたブロック＋ラベル（名前・NDVI値）。
- Polygon → 範囲サイズで押し出した半透明ブロック（重心に配置）。

ハンズオン② 手順

1. 屋外に出て、周囲の安全を確認する。
2. 配布URL（ `.../02_ar/app/` ）を開く。
3. 「開始」を押し、iPhoneは「動作と方向」「カメラ」「位置情報」を許可する。
4. 画面上部にGPSが出たら、その場で一回転して観測ポイントを確認する。
5. 近づく・離れるで、距離に応じて大きさが変わることを見る。

出ないとき：屋内はGPSが定まらない。北がずれるときは端末を8の字に振る。

実装の限界と次の一手

- ポリゴンは重心に「範囲サイズで押し出したブロック」を置いている。厳密な頂点形状ではない。
- 正確なフットプリントは、`gps-new-camera` の `latLonToWorld()` で各頂点をワールド座標へ変換し、`three.js` のカスタムジオメトリで描く。
- 位置精度は数メートル。cm単位が要る用途では、次に触れる VPS を検討する。

DEVICE / PERMISSION

iOS と Android の差異まとめ

項目	iOS (Safari)	Android (Chrome)
方位	webkitCompassHeading（真北基準）	deviceorientationabsolute（360 – alpha）
センサー権限	requestPermission（要ユーザージェスチャ）	原則ダイアログ不要
WebXR immersive-ar	利用できない	対応機で可
共通	HTTPS必須・屋外前提・地磁気のキャリブレーション	同左

パーミッション設計

- 位置・カメラ・モーション（方位）は別々の許可。まとめて事前に説明しておく。
- iOSはボタントップ内で `requestPermission()` を呼ぶ。読み込み時の自動要求は拒否される。
- 一度拒否されると再要求は難しい（ブラウザ設定）。最初の導線で確実に許可を取る。
- すべて HTTPS 前提。localhost は同一端末のみで、スマホからは届かない。

HIGH PRECISION

位置合わせをどう高精度化するか — VPS

- GPS + 方位だけでは数メートル程度。cm～dm精度が要る場合は **VPS（Visual Positioning System）** を使う。
- 仕組み：事前に作った3Dマップ（点群 / Street View由来）に、カメラ画像の特徴点を照合し、コンピュータビジョンで端末の位置・姿勢（6DoF）を算出する。**幾何的なマッチング** が本体。
- 代表例：
 - **Google ARCore Geospatial API**：Street View由来の3D点群にニューラルネットで照合。Android中心。
 - **Immersal**：事前マップに対しカメラ画像から緯度・経度・高度を返す。Webブラウザ向けSDKあり。

8th Wall（Niantic）のVPSはcm級だったが、OSS化にVPSは含まれず2027年2月に停止予定。ロケーションベースWebARの土台は再編途上。

Webで「画像認識 × AR」を成立させるには

- **Immersal の Web VPS**（`vps-for-web`）：事前マッピング（Mapperアプリ / 360カメラ）→ ブラウザでカメラ画像から緯度経度・姿勢を取得。IMUで向きを追い、連続localizationでSLAM的に追従。iOS/Android両対応・アプリ不要。
- 前提：対象空間を事前に3Dマップ化しておく必要がある（ゼロショットではない）。
- 生駒事例で足りなかった精度は、この方向（事前マップ + カメラ照合）で埋められる。ただし運用コストとマッピング作業が乗る。

「どこまで自作するか」の判断：ラベル表示なら位置情報ベースで十分。空間に正確に固定したいならVPS。

マルチモーダルモデルはVPSの代わりになるか

- 画像認識モデル（VLM）が得意なのは **意味理解**：看板のOCR、建物の種類、風景の分類。
- VPSが返す **cm単位の6DoF位置姿勢は幾何計算の産物** であり、意味理解とは別系統。モデルは metric な姿勢を返さない。
- 推論のレイテンシ・コストの面でも、リアルタイムの姿勢追跡には不向き。
- 現実的な役割分担：**幾何的な位置決め = VPS、粗い場所推定・意味づけ（OCR・シーン分類） = VLM。**

「モデルを噛ませれば高精度になる」わけではない、という切り分けを押さえておく。

WRAP-UP

まとめ — 本日追ったデータの流れ

- センサー（GPS・地磁気・カメラ）→ Web API → 座標 → 描画、の一連を自分で通した。
- 方位は生値のままでは使えず、OS差と補正が挟まる。
- 位置情報ベースARは数メートル精度。用途に足りるかで、マーカー／VPSと切り分ける。

次回（第4回）

点群・3DGSのWeb可視化

- 3Dフォーマット（3D Tiles / glTF / Potree / COPC）と CesiumJS・Re:Earth の位置づけ。
- CloudCompare → PotreeConverter → deck.gl のパイプライン。
- 3D Gaussian Splatting の現状と入口。

Q&A

実機・ネットワークの都合で当日動かない箇所があれば、[ここで確認する](#)。